Introducción a los Test Unitarios en Cells (BBVA)

Los test unitarios son una parte esencial del desarrollo profesional de software.
Su objetivo es comprobar que cada unidad (función, clase o componente) del código funcione correctamente de forma aislada.
En entornos empresariales como BBVA, el uso de pruebas unitarias es fundamental para asegurar calidad, estabilidad y evitar errores en producción.

¿Qué es un test unitario?

Un test unitario verifica el comportamiento de una pequeña parte del código.
Por ejemplo, una función que suma dos números o un método que formatea una fecha.
Cada test debe probar **"una sola cosa"** y dar un resultado claro: pasa ✅ o falla ❌.

¿Por qué son importantes?

- Permiten detectar errores rápidamente antes de desplegar el código.
- Facilitan refactorizar sin miedo a romper algo.
- Documentan cómo debería comportarse una función o componente.
- Son parte esencial del proceso de integración continua (CI/CD) que usamos en el banco.

Herramientas usadas en Cells

En los proyectos de Cells, los test unitarios usan principalmente:
- **@open-wc/testing** → entorno de pruebas moderno para Web Components.
- **Mocha** → ejecuta los tests.
- **Chai** → provee funciones de aserción como `assert.equal()` o `assert.isTrue()`.
- **Sinon** → permite espiar funciones o simular comportamientos (spies y stubs).

Estructura básica de un test unitario

Un test unitario en Cells suele seguir esta estructura:

```
test('debería mostrar las tarjetas cuando hay categorías por moneda', () => {
  el.hasCurrenciesCategories = true;
  const spy = sinon.spy(el._serviceFinancialOverview, 'getCardsFinancialOverviewCategories');
  el._getData();
  assert.isTrue(spy.called);
});
```

✅ **Qué hace:**
- Se configura el estado del componente (`el.hasCurrenciesCategories = true`).
- Se espía una función para saber si fue llamada (`sinon.spy()`).
- Se ejecuta la función que debería dispararla (`el._getData()`).
- Finalmente, se valida con `assert.isTrue(spy.called)` que efectivamente fue llamada.

Buenas prácticas

1. Prueba una sola cosa por test.
2. Nombra tus tests de forma clara y descriptiva.
3. Usa mocks y spies solo cuando sea necesario.
4. Asegúrate de cubrir tanto los casos esperados como los de error.
5. No dependas de servicios externos ni APIs reales.

Cómo ejecutar los tests

Normalmente se ejecutan con:
cells lit-component:test

Esto ejecuta Mocha y genera un reporte.
En algunos proyectos se configura también una herramienta de cobertura como "**Istanbul (nyc)**" para ver qué partes del código están sin probar.

Recursos recomendados

- [Guía oficial de @open-wc/testing](https://open-wc.org/docs/testing/)
- [Documentación de Mocha](https://mochajs.org/)
- [Documentación de Chai](https://www.chaijs.com/)
- [Sinon.js (spies y stubs)](https://sinonjs.org/)
- [Testing Web Components - Google Developers]
(https://developers.google.com/web/fundamentals/web-components/customelements#testing)